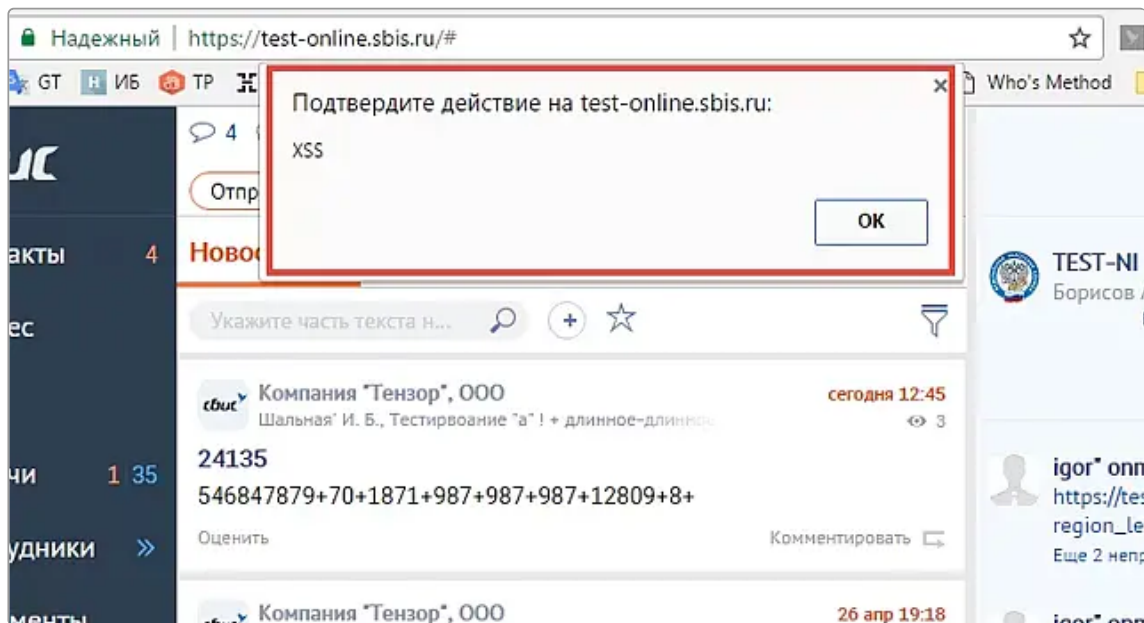


XSS

Описание проблемы

Уязвимость — недостаточная фильтрация данных при отображении в браузере пользователя. Как результат – атака типа XSS (Cross Site Scripting, межсайтовое выполнение запросов). Суть атаки заключается в выполнении произвольного кода в браузере пользователя.



Примеры уязвимостей – внедрение тега, атрибута или ссылки.

Внедрение тега

Пусть есть шаблон следующего вида:

```
<div> hello {{=it.fullname}} </div>
```

где **it.fullname** – полное имя пользователя, которое он сам же и задает. Если пользователь задаст имя

Василий `<script>alert('XSS')</script>` Пупкин

то при рендеринге шаблона всплывет окно с текстом 'XSS', что свидетельствует о наличии уязвимости.

```
1 <!-- Небезопасно -->
2 <div> hello Василий <script>alert('XSS')</script> Пупкин </div>
```

Проблема в том, что угловые скобки подставляются как есть. Решение заключается в их преобразовании в соответствующие html сущности `<` и `>`, что скажет браузеру отрисовать их, но не расценивать как html-тег.

Внедрение атрибута

Пусть есть шаблон следующего вида:

```

```

где описание задается пользователем (в данном случае задается как есть).

Если пользователь задаст описание

```
foo" onmouseover=alert('XSS') "bar
```

то при наведении на картинку всплывет сообщение с текстом 'XSS', что свидетельствует о наличии уязвимости.

```
1 <!-- Небезопасно -->
2 
```

Проблема в том, что двойные кавычки подставляются как есть, что позволяет манипулировать атрибутами html-тега. Решение заключается в преобразовании двойных кавычек в соответствующую html-сущность ";.

Внедрение ссылки

Допустим, есть шаблон следующего вида:

```
<a href="{=it.link}"> foo </a>
```

где ссылка задается пользователем.

Если пользователь задаст ссылку

```
javascript:alert('XSS')
```

то при клике на ссылку всплывет сообщение с текстом 'XSS', что свидетельствует о наличии уязвимости.

```
1 <!-- Небезопасно -->
2 <a href="javascript:alert('XSS')"> foo </a>
```

Проблема в том, что ссылкой может быть также JavaScript-обработчик. Чтобы исключить подобную ситуацию нужно проверять корректность ссылки при задании ее пользователем, либо при выводе на страницу.

Последствия эксплуатации

В результате успешной атаки можно:

1. получить доступ к учетной записи пользователя, скомпрометировав сессионные cookie;
2. выполнить произвольный JavaScript-код, например, отправить от имени пользователя запрос к БЛ;
3. вызывать методы СБИС Плагина;
4. если у пользователя уязвимая версия браузера, то XSS может позволить выполнить произвольный код на компьютере.

Как исправить?

Logicless-шаблонизатор

Использовать платформенный Logicless-шаблонизатор, который по умолчанию обеспечивает защиту от XSS.

Вышеописанные шаблоны будут выглядеть следующим образом:

```
1 <!-- Безопасно -->
```

```

2 <div> hello {{fullname}} </div>
3 

```

Важно!

Декоратор unescape снимает экранирование. Использовать его рекомендуется лишь в исключительных ситуациях.

```

1 <!-- Небезопасно -->
2 {{ myvar|unescape }}

```

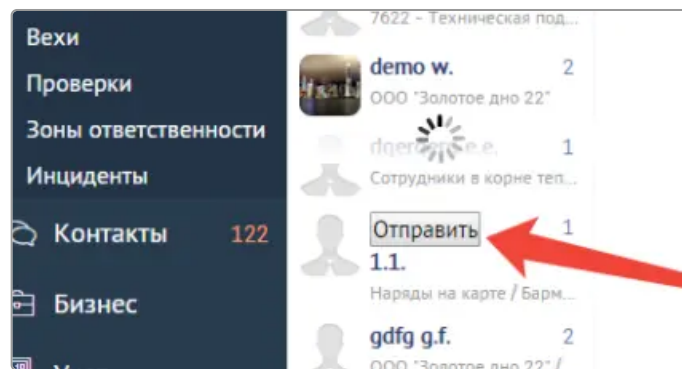
Если на клиент должна прийти верстка, то нужно использовать декоратор sanitize. В таком случае разметка "очистится" от вредоносных тегов (специальные символы при этом экранированы не будут).

```

{{ '<span>hello!</span><script>alert(1)</script>'|sanitize }} → <span>hello!
</span>

```

Не стоит злоупотреблять декоратором sanitize без лишней необходимости. В случаях, где не нужна верстка, может быть проведена атака типа html-инъекция и добавлены элементы интерфейса самим атакующим.



[Документация по logicless-шаблонизатору](#)

doT.js

Важно!

Шаблонизатор doT.js объявлен устаревшим, и всю разработку следует вести на Logicless-шаблонизаторе.

Если всё-таки используется doT.js шаблонизатор, то нужно экранировать все специальные символы в соответствующие html-сущности. Для этого нужно использовать предикат!

```

1 <!-- Безопасно -->
2 <div> hello {!!it.fullname}} </div>
3
4 <!-- Небезопасно -->
5 

```

```

6
7 <!-- Небезопасно -->
8 <div> hello {{=it.fullname}} </div>
9 

```

Допускается такой вариант экранирования:

```

1 <!-- Безопасно -->
2 <div> hello {{=escapeHtml(it.fullname)}} </div>
3 

```

Чтобы не менять все вхождения оператора "=" для конструкций шаблонизатора, допускается в JS-модуле компонента в массиве зависимостей при подключении файла вёрстки принудительно установить экранирование всех значений:

```

1 define('SBIS3/MyArea/MyControl', [
2   'Lib/Control/CompoundControl/CompoundControl',
3
4   // Установили экранирование всех спецсимволов
5   'html!encode=true?SBIS3/MyArea/MyControl'
6 ],
7   ...
8 );

```

[Документация по doT.js](#)

 Настоятельно рекомендуется экранировать все значения.

Sanitizer

В случае если на клиент должна прийти верстка, то нужно обязательно исключить из нее все вредоносные теги и атрибуты. Для этого нужно воспользоваться платформенным санитизатором Sanitize:

```

1 // Безопасно
2 define('SBIS3.foo.bar', [
3   'Core/Sanitize'
4 ], function(sanitizer) {
5   ...
6   var safe_text = sanitizer(unsafe_text);
7   ...
8 })

```

Или же, в современной версии фреймворка, необходимо использовать декоратор [Markup](#), который принимает вёрстку в формате JsonML:

```

1 <!-- Безопасно -->
2 <Controls.markup:Decorator value="{{ json }}" />

```

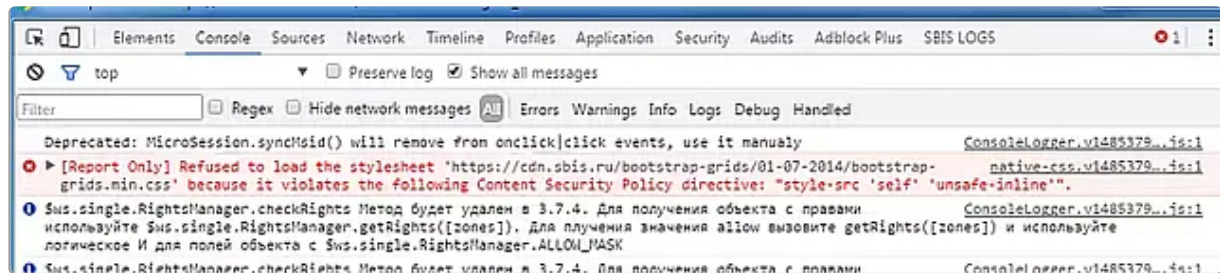
 Декоратор Markup поддерживает расширение списка разрешённых html-тегов, эту опцию стоит использовать только в крайнем случае, и по согласованию с отделом тестирования безопасности приложений.

CSP и mixed-content

Описание проблемы

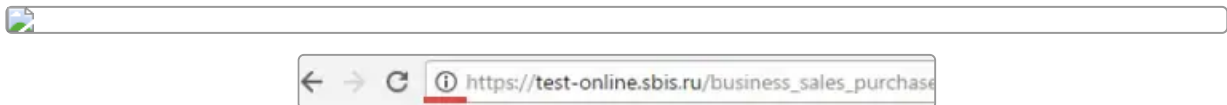
Content Security Policy (CSP, политика защиты контента) – это механизм обеспечения безопасности, с помощью которого можно защищаться от атак с внедрением контента, например, межсайтового скриптинга (XSS, cross site scripting). CSP описывает безопасные источники загрузки ресурсов, устанавливает правила использования встроенных стилей, скриптов, изображений. Загрузка с ресурсов, не входящих в "белый список", блокируется.

Пример 1.



Программист реализовал подгрузку ресурсов с домена cdn.sbis.ru. Это неправильно, ресурсы нужно брать с сервиса /cdn/.

Пример 2.



Использование небезопасного соединения для подгрузки ресурсов. Вследствие этого, браузер перестает считать соединение защищенным, соответствующим образом уведомляет пользователя об этом (пропадает замок в адресной строке). Злоумышленник имеет возможность подменять ресурсы.

Результат эксплуатации

Возможность подмены ресурсов: изображений, стилей, скриптов. Злоумышленник имеет возможность выполнить произвольный код в браузере пользователя.

Что делать

1. Всегда использовать относительные ссылки. Не завязываться на определенный протокол, на определенный стэнд.

```
1 <!-- Безопасно -->
2 <a href=page.html>На главную</a>

1 <!-- Небезопасно -->
2 <a href=https://online.sbis.ru/page.html>На главную</a>
```

2. Не использовать ссылки на сторонние ресурсы без дополнительной необходимости. При наличии такой необходимости уведомлять отдел тестирования безопасности для коррекции правил CSP.